Oracle 19c SQL

Lab Exercise

Retrieving Data Using the SQL SELECT Statement

1. Display the structure of the Employees table.

2. Display the structure of the Departments table. Select all the rows from the Departments table.

3. Display the structure of the Employees table. Write a query to retrieve the employee ID, first name, last name, job ID, hire date, department ID, and salary from the Employees table. Save the query to script file.

4. Look at the following SELECT statement. Can you identify any coding errors?

   SELECT Employee_ID, First_Name, Job_ID

   Salary x 0.1 Raised Salary

   FROM Employees;

   Execute the above statement and observe the error messages displayed.

5. Write a query to display the unique department numbers and job IDs from the Employees table.

6. Write a query to display the last name, job ID, salary and annual compensation of each employee. The annual compensation is to be calculated by adding a bonus of $500 to monthly salary, and then multiplying the result by 12. The query should combine the last name and job ID to make a single column named as 'Employees'. Give an alias 'Annual Compensation' for the calculated column.

7. Modify the file into the SQL Developer and make the following modifications:

   - Name the Employee_ID column as 'Emp_No'
   - concatenate first name and last name of employees separated by a space. Name the column as 'Emp_Name'
   - Name Department_ID column as 'Dept_No' The query should display the result as shown below.

8. Write a query to display the following output for all employees:

   Den working as PU_MANin Dept No30earns $ 11000

9. Write a query to display the following text using q operator.

   Oracle's quote operator

Lab Exercise

Restricting and Sorting Data

1. Write a query to display details of employees whose job ID is 'ST_CLERK'.
2. Write a query to display last name, job ID, department number and salary of employees whose salary is greater than $14,000.
3. Write a query to display last name, department number and hire date of employees hired between 01-JAN-1995 and 31-DEC-1999. Sort the query in descending order of hire date.
4. Write a query to display the employee ID, first name and manager ID of employees whose manager ID is 100, 108, 148 or 120.
5. Write a query to retrieve the last name and hire date of employees hired in 1997.
6. Write a query to retrieve the employee ID, last name and job ID of employees whose job ID contains the string 'SH_'.
7. Write a query to retrieve the first name, job ID and Department number of employees who do not have a manager.
8. Write a query to retrieve the employee ID, last name, salary and commission of all employees who earn a commission. Sort the query result in descending order of salary.
9. Write a query to retrieve the last name, department number, job ID and salary of employees if, the job ID is 'SA_MAN' or 'SA_REP', and if the employee earns a salary more than $10,000.
10. Write a query to retrieve the last name, job ID, department number and salary for employees whose department ID is 50 or whose job ID contains the string 'MAN'.
11. Write a query to display the details of employee whose last name is 'ernst'.

Lab Exercise

Using Single-Row Functions and Conversion Functions

1. Write a query to display:
   - The last name in uppercase
   - The length of the last names
   - Job IDs starting from the fourth character.

2. Write a query to display employee ID, first name, salary and salary increased by 10%, which is expressed as a whole number. Name the column as 'New_Sal'.

3. Write a query to display the first name and number of months worked by the employees. Round the number of months worked to the closest whole number. Sort the result by the number of months employed.

4. Write a query to display the employee ID, hire date, and salary. The hire date should appear as (For E.g.) 'Seventh of June 1994 12:00:00 AM'. The salary should appear as $15,000. Give each column an appropriate label as shown below.

EMPLOYEE_ID	Hired On	Earns
100	Seventeenth of June 1987 12:00:00 AM	$24,000
101	Twenty-First of September 1989 12:00:00 AM	$17,000
102	Thirteenth of January 1993 12:00:00 AM	$17,000
103	Third of January 1990 12:00:00 AM	$9,000
104	Twenty-First of May 1991 12:00:00 AM	$6,000
105	Twenty-Fifth of June 1997 12:00:00 AM	$4,800
106	Fifth of February 1998 12:00:00 AM	$4,800

5. Write a query to display the employee ID, hire date and salary review date. The salary review date is the first Friday after a year of service. Name the column as 'Salary Review Dt.'. The salary review dates should appear as 'Friday, 31st of July, 2000'.

6. Write a query to retrieve the last name and hire date of employees hired before 1994. The query should return the same result even if the query is run in 1994 or now.

7. Write a query to retrieve the employee ID, salary, commission, and annual salary, which is calculated by multiplying salary by 12, and then adding the commission percentage to the result. Annual salary is to be calculated for all employees irrespective of whether they earn a commission or not.

8. Write a query to retrieve the employee ID, job ID and salary of all employees. The query should calculate increase in salary on the following basis:

- If job ID is ‘SA_REP’, salary increase is 10%
- If job ID is ‘ST_CLERK’, salary increase is 15%
- If job ID is ‘IT_PROG’, salary increase is 20%
- For all other job IDs, there is no salary increase.

Perform the above query using CASE expression as well as DECODE function.

Lab Exercise

Reporting Aggregated Data Using the Group Functions

1. Write a query to display the maximum, minimum, sum, and average salaries for each Job type.
2. Write a query to display the number of employees belonging to the same department numbers.
3. Write a query to display the manager ID and the highest paid employee for that manager. Exclude employees who do not have a manager.
4. Write a query to display the job ID and total salary for each job that has total salary exceeding $13000.

Lab Exercise

Displaying Data from Multiple Tables Using Joins

1. Write a query to retrieve the first name and last name of employees along with their respective department numbers and department names.

2. Write a query to retrieve the last name, department name, city, and commission earned of all those employees who earn a commission.

3. Write a query to retrieve the first name, last name, salary and job titles of employees belonging to department numbers 100 and 30.

4. Write a query to retrieve the employee's last names, department numbers, and department names, including those departments that do not have employees. The result should be as follows:

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Grant	50	Shipping
Whalen	10	Administration
Hartstein	20	Marketing
Fay	20	Marketing
Mavris	40	Human Resources
Baer	70	Public Relations
Higgins	110	Accounting
Gietz	110	Accounting
		NOC
		Manufacturing
		Government Sales
		IT Support
		Benefits
		Shareholder Services

5. Write a query to retrieve the names of each employee and their managers. The result should be shown as follows

Subordinates-Superiors
Hartstein reports to: King
Zlotkey reports to: King
Cambrault reports to: King
Errazuriz reports to: King
Partners reports to: King
Russell reports to: King
Mourgos reports to: King
Vollman reports to: King
Kaufling reports to: King
Fripp reports to: King
Weiss reports to: King
Raphaely reports to: King
De Haan reports to: King
Kochhar reports to: King

6. Write a query to retrieve the last names and department names of employees who have superiors.

7. Write a query to retrieve the names of employees and their managers.

8. Write a query to retrieve the last name, and department name of all employees even if the employees have not been assigned to any department.

9. Write a query to retrieve the employee IDs, department numbers, and department names even if employees have not been appointed to some of the existing departments. The result should be as shown below.

EMPLOYEE_ID	DEPARTMENT_ID	DEPARTMENT_NAME
199	50	Shipping
200	10	Administration
201	20	Marketing
202	20	Marketing
203	40	Human Resources
204	70	Public Relations
205	110	Accounting
206	110	Accounting
		NOC
		Manufacturing
		Government Sales
		IT Support
		Benefits
		Shareholder Services

10. Write a query to retrieve the employee ID, department number, and department names of those employees who have not been assigned to any department, and those departments to which employees have not yet been assigned. The result should be as shown below.

	DEPARTMENT	DEPARTMENT
113	100 Financ	
112	100 Financ	
111	100 Financ	
110	100 Financ	
109	100 Financ	
108	100 Financ	
206	110 cco ntin	
205	110 cco ntin	
1 8		
	NOT	
	Man fact ng	
	Go mm nt as	
	T S o	

Lab Exercise

Using Subqueries to Solve Queries

1. Write a subquery to display the last name and salary of employees in the same Department as Nancy.
2. Write a query to retrieve the employee numbers, last names and salaries of all Employees whose salary is greater than the average salary.
3. Display the last name, department number and salary of all employees whose location ID is 1400.
4. Write a query to display the last name, department number, and salaries of all employees who report to Hunold.
5. Write a query to display last name, job ID and manager ID of employees belonging to the marketing department.
6. Write a query to display the last names, department ID, and salaries of all employees whose department ID and salary match the department ID and salary of all employees Who earn a commission.

Lab Exercise

Using the Set Operators

1. The HR department needs a list of department IDs for departments that do not contain the job ID ST_CLERK. Use set operators to create this report.
2. The HR department needs a list of countries that have no departments located in them.Display the country ID and the name of the countries. Use set operators to create this report.
3. Produce a list of jobs for departments 10, 50, and 20, in that order. Display the job ID and department ID using set operators.
4. Create a report that lists the employee ID and job ID of those employees who currently have a job title that is the same as their job title when they were initially hired by the company (that is, they changed jobs but have now gone back to doing their original job).

5. The HR department needs a report with the following specifications:
   - The last name and department ID of all employees from the EMPLOYEES table,regardless of whether or not they belong to a department
   - The department ID and department name of all departments from the DEPARTMENTS
6. table, regardless of whether or not they have employees working in them.

Write a compound query to accomplish this.

Lab Exercise

Manipulating Data

To work with the **OfficeStaff** table. The structure of the table is as follows:

Name	Null?	Type
STAFF_ID	NOT NULL	NUMBER(4)
F_NAME		VARCHAR2(20)
L_NAME		VARCHAR2(20)
MAIL_ID		VARCHAR2(10)
SALARY		NUMBER(7,2)

The rows to be inserted are given below.

Staff_ID	F_Name	L_Name	Mail_ID	Salary
101	Jack	Skinner	JSkinner	2500
102	Morgan	Sheen	MSheen	4950
103	Tom	Smart	TSmart	7510
104	Susan	Bright	SBright	3220
105	Judy	Brown	JBrown	6900

1. From the sample data given above, insert the first row into the **OfficeStaff** table without listing the columns in the INSERT clause.

2. Insert the second row from the sample data into the **OfficeStaff** table by explicitly listing the columns in the INSERT clause. Confirm the creation of rows.

3. Write an INSERT statement to populate the OfficeStaff table. The INSERT statement should be such that it prompts you to enter the required values. To produce values for the **Mail_ID** column, concatenate the first letter of first name with the last names.

4. Now run the script file to insert the next two rows of sample data. Make the data inserted permanent. Confirm the rows inserted into the **OfficeStaff** table.

5. Increase 'Susan Bright's' salary by 10%.

6. Remove the details of 'Morgan Sheen' from the **OfficeStaff** table. Confirm the changes made. Commit the changes made.

7. Insert the last row of the sample data. Mark a savepoint immediately after inserting the row.

8. Delete all the rows from the **OfficeStaff** table. Confirm that the table is empty. Now,rollback the DELETE statement, but ensure that the earlier INSERT statement is not discarded. Make the transaction permanent.

9. Write a query to update the salary of the employees working in department number 10 by 10% and using the returning clause to print the updated value.

Lab Exercise

Using DDL Statements to Create and Manage Tables

1. Create a table **Dept_details** based on the details given in the following chart. Save the syntax in a script named LabEx12_01.sql. Execute the script and confirm that the table has been created.

Column Name	deptID	deptName
Key Type		
Nulls/Unique		
FK Table		
FK Column		
Data type	Number	VARCHAR2
Length	5	30

Name	Null?	Type
DEPTID		NUMBER(5)
DEPTNAME		VARCHAR2(30)

2. Populate the **Dept_details** table with data from Departments table. Insert only the columns that you need.

3. Create a table **Emp_details** based on the following chart.

	empID	empFirstName	empLastName	DeptID	Salary
Key Type					
Nulls/Unique					
FK Table					
FK Column					
Data Type		VARCHAR2	VARCHAR2	Number	Number
Length	9	15	15	5	8,2

Name	Null?	Type
EMPID		CHAR(9)
EMPFIRSTNAME		VARCHAR2(15)
EMPLASTNAME		VARCHAR2(15)
DEPTID		NUMBER(5)
SALARY		NUMBER(8,2)

4. Modify the **empFirstName** and **empLastName** columns in the **Emp_details** table to allow for longer names.

Name	Null?	Type
EMPID		CHAR(9)
EMPFIRSTNAME		VARCHAR2(30)
EMPLASTNAME		VARCHAR2(30)
DEPTID		NUMBER(5)
SALARY		NUMBER(8,2)

5. Create a table **emp2** based on the structure of employees table. Include the columns **employee_id**, **first_name**, **last_name**, **department_id**, **job_id**, and **salary** into the **Emp2** table.

Name	Null?	Type
EMPID		CHAR(9)
EMPFIRSTNAME		VARCHAR2(30)
EMPLASTNAME		VARCHAR2(30)
DEPTID		NUMBER(5)
SALARY		NUMBER(8,2)

6. Drop the **Emp_details** table.
7. Rename the table **Emp2** as **Emp_details**.
8. Drop the column **first_name** from the table **Emp_details** and see the structure of the table

Name	Null?	Type
EMPLOYEE_ID		NUMBER(6)
LAST_NAME	NOT NULL	VARCHAR2(25)
DEPARTMENT_ID		NUMBER(4)
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)

9. Make the **job_id** column of **Emp_details** table UNUSED and confirm the modification.
10. Drop all the unused columns from **Emp_details** table, confirm the modification.

Name	Null?	Type
EMPLOYEE_ID		NUMBER(6)
LAST_NAME	NOT NULL	VARCHAR2(25)
DEPARTMENT_ID		NUMBER(4)
SALARY		NUMBER(8,2)

11. Create a table **New_Emp** having employee details, and hire date as TIMESTAMP datatype.

Name	Null?	Type
EMPID		NUMBER(5)
FIRSTNAME		VARCHAR2(25)
LASTNAME		VARCHAR2(25)
DEPTID		CHAR(5)
SALARY		NUMBER(8,2)
HIREDATE		TIMESTAMP(6)

12. Modify the table **New_Emp** with hiredate as TIMESTAMP WITH TIMEZONE datatype. View the structure of the table.

Name	Null?	Type
EMPID		NUMBER(5)
FIRSTNAME		VARCHAR2(25)
LASTNAME		VARCHAR2(25)
DEPTID		CHAR(5)
SALARY		NUMBER(2)
HIREDATE		TIMESTAMP(6) WITH TIME ZONE

13. Try the following:

    a) Create table test with column specified below:

    ```
    x NUMBER
    ```

    b) Select all the tables from present schema.

    c) Drop the Test table from schema.

    d) Select all the tables from current schema and see the change.

    e) Display the contents of RecyleBin

    f) Write the command to reinstate the table.

    g) Select all the tables from current schema.

    h) Drop the table completely without flashback feature or drop it permanently.

    i) What command is used to remove the table from Recycle Bin after the table is dropped.

Lab Exercise

Creating Other Schema Objects

1. Create a sequence that can be used with primary key column of the **new_dept** table. The sequence should start with 200 and have a maximum value of 1000. Name the sequence as **NEW_DEPT_ID_SEQ** and the sequence should increment by 20.

2. Display the information about your sequence- the sequence name, maximum value, increment size and the last number.

SEQUENCE_NAME	MAX_VALUE	INCREMENT_BY	LAST_NUMBER
DEPARTMENTS_SEQ	9990	10	280
EMPLOYEES_SEQ	1.0000E+27	1	207
LOCATIONS_SEQ	9900	100	3300
NEW_DEPT_ID_SEQ	1000	20	200

3. Insert three rows into the **new_dept** table. Use the sequence you created for the **deptID** column. Add three departments namely, 'Marketing', 'Administration' and 'Education'. Confirm whether the rows have been inserted

DEPTID	DEPTNAME	LOCATIONID
200	Education	10
220	Marketing	20
240	Administration	20

4. Create an index on the foreign key column of the **new_emp** table.

5. Display the indexes and the uniqueness that exist in data dictionary table for the **new_emp** table.

INDEX_NAME	TABLE_NAME	UNIQUENES
EMP_ID_PK	NEW_EMP	UNIQUE
NEW_EMP_DEPT_ID_INDEX	NEW_EMP	NONUNIQUE

6. Create a synonym for the **new_dept** table.

Lab Exercise

Retrieving Data by Using Subqueries

1. Write a query to display the last names, department ID and salaries of all employees whose department ID and salary match the department ID and salary of all employees who earn a commission.

2. Display the last name, department name and salary of all employees whose salary and department name are same as the salary and department name of employees located in location ID 1700.

3. Write a query to display details of employees who have one or more colleagues in their department who joined after them, with higher salaries.

4. Display the details of employees who are not managers.
   [Hint: Use the NOT EXISTS operator. Also, use the NOT IN operator and see the result].

Lab Exercise

Regular Expression Support

1. A string contains three spaces in between. For example 'NEW DELHI'. Write a query to replace the spaces with one. The output displayed should be as NEW DELHI
2. Write a query, which extracts a part of a string. The regular expression searches for a comma followed by a space; then zero or more characters that are not commas, as indicated by [^,]*; and lastly looks for another comma.
3. The pattern will look similar to a comma-separated values string.

   ('first part, second part, third part')
4. Retrieve any values from ename that start with J, followed by any letter, then N or M,then any letter, then S from **Emp** table